University of Minnesota custom Azure course

Lab 2: Build an Archival Landing Zone

Participant lab guide

Session 2: Long-Term Archival, Data Movement at Scale, and Hybrid Identity

Session: Day 1, Session 2, Block 4 (75 minutes: 55 lab, 20 debrief)

Format: Individual, hands-on, in your own lab resource group

Outcome: A working immutable, tagged, tiered, ABAC-governed landing zone and a validation record

All data in this lab is synthetic. No real University of Minnesota data is used or required.

Contents

1. Lab overview 3
1.1 Learning objectives 3
1.2 Timing 3
1.3 Prerequisites 4
2. Part 0: sign in and set your variables (5 minutes) 4
3. Part 1: deploy the storage account with the security baseline (8 minutes) 5
4. Part 2: immutable container, tagged upload, integrity manifest (10 minutes) 6
5. Part 3: archive one object and start the rehydration request (4 minutes) 7
6. Part 4: lifecycle rule targeted at the RetentionClass tag (6 minutes) 8
7. Part 5: conditional read access with ABAC (14 minutes) 9
8. Part 6: revisit rehydration, validate integrity, clean up (8 minutes) 10
9. Debrief questions 11
10. Worksheet 12
11. Reference 12

1. Lab overview

In Session 2 you saw the century-proof landing zone: an archival storage account whose contents cannot be altered or deleted for a defined period, with access granted by a single attribute-based condition instead of a tree of ACLs. In this lab you build that landing zone yourself, step by step, in your own lab resource group.

You will create a standard general-purpose v2 storage account with a flat namespace and zone redundancy, add a container with an unlocked time-based retention policy, enable blob versioning, upload a synthetic satellite dataset with blob index tags and an integrity manifest, add a lifecycle rule that moves permanent data to the Archive tier based on creation time, and grant yourself read access through an ABAC condition that only allows reads where the blob's `ResearchUnit` tag matches. You will also submit a rehydration request early so that by the debrief you have felt Archive latency firsthand.

1.1 Learning objectives

- Deploy a storage account with the course security baseline: HTTPS only, TLS 1.2 minimum, anonymous access off, shared key access off, versioning on.
- Configure a container-level time-based retention policy and prove that it blocks overwrite and delete, even for an account Contributor.
- Explain why last access time tracking stays off on an account that carries immutability policies, and why the lifecycle rule therefore keys on creation or modification time.
- Apply blob index tags at upload time and target them from a lifecycle rule.
- Create an Azure role assignment with an ABAC condition and validate the effect from the data plane.
- Submit an Archive rehydration request and observe its status over time.

1.2 Timing

Part	Activity	Minutes
0	Sign in and set your variables	5
1	Deploy the storage account with the security baseline	8
2	Create the immutable container and upload the dataset with tags and a manifest	10
3	Archive one object and start the rehydration request (long-running; started early on purpose)	4
4	Add the lifecycle rule targeted at the RetentionClass tag	6
5	Grant conditional read access with ABAC and validate it	14
6	Revisit the rehydration request, run the integrity validation, and clean up	8

Part	Activity	Minutes
	Debrief (whole room)	20

1.3 Prerequisites

- Your lab resource group (`rg-lab-NN`) with Contributor and User Access Administrator, assigned by the instructor.
- Azure CLI 2.70 or later with the Bicep CLI (`az bicep install`), or Azure Cloud Shell, which has everything.
- Python 3 for the manifest tools (Cloud Shell has it).
- The lab files folder: `lab-solutions/s02-archival-landing-zone` (Bicep for the optional as-code path) and `scripts/manifest` (the manifest tools).

> **Conventions**
>
> Commands are shown for bash. Replace `NN` with your lab number and `<suffix>` with your initials plus two digits in lowercase (for example `ajs07`); storage account names must be globally unique.
>
> Every part ends with a **Checkpoint**. Do not move on until the checkpoint matches; the facilitator uses the same checks.
>
> Where a portal path and a CLI command are both given, use whichever you prefer; the result is the same resource.

2. Part 0: sign in and set your variables (5 minutes)

1. Open a terminal (or Cloud Shell) and sign in: `az login`. Select the course subscription if prompted.
2. Confirm you are in the right subscription and resource group:

```
az account show --query "{subscription:name, user:user.name}" -o table
az group show -n rg-lab-NN --query "{name:name, location:location}" -o table
```

Set the variables used by every later command. Keep this terminal open; if you open a new one, run these again.

```
export RG=rg-lab-NN
export LOCATION=$(az group show -n $RG --query location -o tsv)
export SUFFIX=<suffix>
export SA=stlab02$SUFFIX
export ME=$(az ad signed-in-user show --query id -o tsv)
echo "Account: $SA  Region: $LOCATION  Me: $ME"
```

> **Checkpoint 0**

> The echo line prints a storage account name of 24 characters or fewer, all lowercase letters and digits, and a GUID for your object ID.

3. Part 1: deploy the storage account with the security baseline (8 minutes)

The archive pattern from Session 1: standard general-purpose v2, flat namespace (because version-level WORM, versioning, and object replication need it), zone redundancy, Hot default tier (the lifecycle rule handles tiering), and the security baseline on every account.

Portal path

1. Portal > Storage accounts > **Create**. Resource group `rg-lab-NN`, name `stlab02<suffix>`, region your lab region, performance **Standard**, redundancy **Zone-redundant storage (ZRS)**.
2. Advanced tab: **Require secure transfer** on; **Allow enabling anonymous access on individual containers** off; **Enable storage account key access** off; **Minimum TLS version** 1.2; **Enable hierarchical namespace** off.
3. Data protection tab: **Enable versioning for blobs** on; **Enable soft delete for blobs** 7 days; **Enable soft delete for containers** 7 days; leave point-in-time restore off.
4. Tags tab: `ResearchUnit` = `PGRL`, `DataClass` = `SatelliteTelemetry`, `RetentionClass` = `Permanent`, `Owner` = your lab number. Review and create.

CLI path

```
az storage account create -n $SA -g $RG -l $LOCATION \
  --kind StorageV2 --sku Standard_ZRS --access-tier Hot \
  --https-only true --min-tls-version TLS1_2 \
  --allow-blob-public-access false --allow-shared-key-access false \
  --enable-hierarchical-namespace false \
  --tags ResearchUnit=PGRL DataClass=SatelliteTelemetry RetentionClass=Permanent Owner=lab-
NN

az storage account blob-service-properties update -n $SA -g $RG \
  --enable-versioning true \
  --enable-delete-retention true --delete-retention-days 7 \
  --enable-container-delete-retention true --container-delete-retention-days 7
```

Do **not** enable last access time tracking on this account. Immutable storage cannot be combined with it, and Part 2 adds an immutability policy.

> **Checkpoint 1**
> The command below prints `Standard_ZRS`, `TLS1_2`, `false`, `false`, and `true` for versioning.

```
az storage account show -n $SA -g $RG \
  --query "{sku:sku.name, tls:minimumTlsVersion, publicBlob:allowBlobPublicAccess,
sharedKey:allowSharedKeyAccess, hns:isHnsEnabled}" -o table
az storage account blob-service-properties show -n $SA -g $RG --query "isVersioningEnabled"
```

4. Part 2: immutable container, tagged upload, integrity manifest (10 minutes)

4.1 Grant yourself data-plane access for the upload

Contributor is a management-plane role; it does not let you read or write blob data with Microsoft Entra ID. Grant yourself **Storage Blob Data Contributor** on the account for the setup steps. You will remove it in Part 5 so the conditional role can be tested on its own.

```
export SCOPE=$(az storage account show -n $SA -g $RG --query id -o tsv)
az role assignment create --role "Storage Blob Data Contributor" \
  --assignee-object-id $ME --assignee-principal-type User --scope $SCOPE -o table
echo "Waiting 60 seconds for the assignment to propagate"; sleep 60
```

4.2 Create the containers and the unlocked retention policy

The lab uses a one-day interval so your resources can be cleaned up tomorrow. Production uses the retention the data stewards approve, and locks it; the lab leaves the policy unlocked.

```
az storage container create --account-name $SA -n satellite-archive --auth-mode login -o
none
az storage container create --account-name $SA -n manifests --auth-mode login -o none
az storage container create --account-name $SA -n cism-reference --auth-mode login -o none

az storage container immutability-policy create --account-name $SA -c satellite-archive -g
$RG \
  --period 1 --allow-protected-append-writes false -o table
```

Portal equivalent: storage account > Containers > `satellite-archive` > **Access policy** > Immutable blob storage > **Add policy** > Time-based retention, 1 day, **Unlocked**.

4.3 Build a synthetic satellite dataset and its manifest

```
mkdir -p ~/lab02/satellite && cd ~/lab02
for i in $(seq -w 1 12); do head -c $((2*1024*1024)) /dev/urandom >
satellite/noaa19_pass_2026_10_$i.bin; done
python3 <path-to-lab-files>/scripts/manifest/make_manifest.py satellite manifest-2026-
10.json --dataset-id DS-LAB02-$SUFFIX
cat manifest-2026-10.json | head -20
```

The manifest lists every object with its SHA-256 hash and carries a hash of itself. One copy goes with the data; one goes to a separate container (and in production, a separate protected location).

4.4 Upload with blob index tags

```
az storage blob upload-batch --account-name $SA --auth-mode login -d satellite-archive -s
satellite \
  --tags ResearchUnit=PGRL RetentionClass=Permanent DataClass=SatelliteTelemetry -o table
az storage blob upload --account-name $SA --auth-mode login -c manifests -n
satellite/manifest-2026-10.json \
  -f manifest-2026-10.json --tags ResearchUnit=PGRL RetentionClass=Permanent -o none
echo "climate placeholder" > cesm2_note.txt
az storage blob upload --account-name $SA --auth-mode login -c cism-reference -n
cesm2_note.txt \
  -f cesm2_note.txt --tags ResearchUnit=CISM RetentionClass=Reference -o none
```

The `cism-reference` object belongs to a different research unit; it exists so Part 5 has something to be denied.

> **Checkpoint 2**
>
> Twelve objects in `satellite-archive` carry the three tags, and a delete attempt is refused. Run:
>
> `az storage blob list --account-name $SA -c satellite-archive --auth-mode login --include t --query "[0].{name:name,tags:tags}"`
>
> `az storage blob delete --account-name $SA --auth-mode login -c satellite-archive -n noaa19_pass_2026_10_01.bin`
>
> The delete must fail with a 409 stating the operation is not permitted because the blob is immutable due to a policy. Write the first line of the error on your worksheet.

5. Part 3: archive one object and start the rehydration request (4 minutes)

Archive is an offline tier: Standard priority rehydration can take up to 15 hours for objects under 10 GB, and High priority may finish in under an hour. You start the request now so the debrief can discuss the real wait.

```
az storage blob set-tier --account-name $SA --auth-mode login -c satellite-archive -n
noaa19_pass_2026_10_12.bin --tier Archive -o none
az storage blob show --account-name $SA --auth-mode login -c satellite-archive -n
noaa19_pass_2026_10_12.bin \
  --query "{tier:properties.blobTier, archiveStatus:properties.archiveStatus}" -o table

# Request rehydration to Cool at Standard priority (Set Blob Tier rehydrates in place)
az storage blob set-tier --account-name $SA --auth-mode login -c satellite-archive -n
noaa19_pass_2026_10_12.bin --tier Cool --rehydrate-priority Standard -o none
date -u
az storage blob show --account-name $SA --auth-mode login -c satellite-archive -n
noaa19_pass_2026_10_12.bin \
  --query "{tier:properties.blobTier, archiveStatus:properties.archiveStatus,
priority:properties.rehydratePriority}" -o table
```

Write down the time of the request. The blob stays in Archive with `archiveStatus` set to `rehydrate-pending-to-cool` until the service finishes; nothing you do speeds it up except raising the priority to High, which costs more.

> **Checkpoint 3**
> `archiveStatus` shows `rehydrate-pending-to-cool` and `priority` shows `Standard`.

6. Part 4: lifecycle rule targeted at the RetentionClass tag (6 minutes)

The rule moves `Permanent` objects to Archive one day after creation. It targets the blob index tag, not a folder prefix, so it follows the data wherever it is stored in the account. Because immutable storage rules out last access tracking, the condition is creation time.

```
cat > lifecycle.json <<'EOF'
{
  "rules": [
    {
      "enabled": true,
      "name": "permanent-to-archive-by-creation",
      "type": "Lifecycle",
      "definition": {
        "filters": {
          "blobTypes": [ "blockBlob" ],
          "blobIndexMatch": [ { "name": "RetentionClass", "op": "==", "value": "Permanent"
} ]
        },
        "actions": {
          "baseBlob": { "tierToArchive": { "daysAfterCreationGreaterThan": 1 } }
        }
      }
    }
  ]
}
EOF
az storage account management-policy create --account-name $SA -g $RG --policy
@lifecycle.json -o table
```

Portal equivalent: storage account > Data management > Lifecycle management > **Add a rule** > rule scope **Filter blobs with blob index tags**, blob type block blobs, condition **Created** more than 1 day ago, action **Move to archive storage**, filter `RetentionClass == Permanent`.

> **Checkpoint 4**
> `az storage account management-policy show --account-name $SA -g $RG --query "policy.rules[].{name:name,enabled:enabled,tag:definition.filters.blobIndexMatch[0].value}" -o table` shows one enabled rule with tag `Permanent`. The rule runs once a day, so the tier change itself appears tomorrow.

7. Part 5: conditional read access with ABAC (14 minutes)

7.1 Remove the broad data role

Azure RBAC is additive: if you keep Storage Blob Data Contributor, the conditional Reader role cannot restrict anything. Remove the broad role first.

```
az role assignment delete --role "Storage Blob Data Contributor" --assignee $ME --scope
$SCOPE
sleep 45
az storage blob download --account-name $SA --auth-mode login -c satellite-archive -n
noaa19_pass_2026_10_01.bin -f /tmp/test.bin
```

Expected: the download now fails with `AuthorizationPermissionMismatch`. That is your baseline: no data access at all.

7.2 Add Storage Blob Data Reader with a condition

The condition reads: for the blob read action (excluding listing), allow only when the blob's `ResearchUnit` index tag equals `PGRL`. Listing is excluded so you can still browse; reading content is what the tag governs.

```
COND="((!(ActionMatches{'Microsoft.Storage/storageAccounts/blobServices/containers/blobs/read'} AND NOT
SubOperationMatches{'Blob.List'})) OR
(@Resource[Microsoft.Storage/storageAccounts/blobServices/containers/blobs/tags:ResearchUnit<\$key_case_sens
StringEquals 'PGRL'))"

az role assignment create --role "Storage Blob Data Reader" \
  --assignee-object-id $ME --assignee-principal-type User --scope $SCOPE \
  --description "Lab 2 ABAC: read only PGRL-tagged blobs" \
  --condition "$COND" --condition-version "2.0" \
  --query "{role:roleDefinitionName, condition:condition}" -o yaml
sleep 60
```

Portal equivalent: storage account > Access Control (IAM) > Add role assignment > Storage Blob Data Reader > your user > **Conditions (optional)** > Add condition > Add action **Read a blob** > Build expression: attribute source Resource, attribute **Blob index tags [Values in key]**, key `ResearchUnit`, operator StringEquals, value `PGRL`.

7.3 Validate

```
# Listing works (the condition excludes Blob.List)
az storage blob list --account-name $SA -c satellite-archive --auth-mode login --query
"[0:3].name" -o table
# A PGRL-tagged read succeeds
az storage blob download --account-name $SA --auth-mode login -c satellite-archive -n
noaa19_pass_2026_10_01.bin -f /tmp/pgrl.bin -o none && echo "PGRL read: OK"
# A CISM-tagged read is refused
az storage blob download --account-name $SA --auth-mode login -c cism-reference -n
cesm2_note.txt -f /tmp/cism.txt
# Any write is refused (Reader has no write action at all)
```

```
az storage blob upload --account-name $SA --auth-mode login -c manifests -n test.txt -f
cesm2_note.txt
```

Checkpoint 5

Listing succeeds, the PGRL download succeeds, the CISM download fails with `AuthorizationPermissionMismatch`, and the upload fails. If the PGRL download also fails, wait another minute: role assignments can take a few minutes to reach the data plane.

7.4 Think about it

- You granted one role with one condition. How many ACLs on today's file servers does that condition replace for a research unit with 40 shares?
- The condition is evaluated per request against the blob's tags. What process must therefore own the tags, and when must they be applied?

8. Part 6: revisit rehydration, validate integrity, clean up (8 minutes)

8.1 Rehydration status

```
az storage blob show --account-name $SA --auth-mode login -c satellite-archive -n
noaa19_pass_2026_10_12.bin \
  --query "{tier:properties.blobTier, archiveStatus:properties.archiveStatus}" -o table
date -u
```

Most likely it is still pending. Record the elapsed time so far on your worksheet; the facilitator will collect the room's numbers at the debrief tomorrow morning for anyone whose request completes overnight.

8.2 Integrity validation against the manifest

The verifier downloads each online object, hashes it, and compares with the manifest. The archived object is reported as offline, not failed; that is the correct result for an Archive object and is why validation schedules are planned around rehydration.

```
pip install --quiet azure-identity azure-storage-blob 2>/dev/null || true
python3 <path-to-lab-files>/scripts/manifest/verify_manifest.py manifest-2026-10.json \
  --account $SA --container satellite-archive --report validation-$(date -u +%Y%m%d).json
cat validation-*.json | head -12
```

Checkpoint 6

`result` is `PASS`, `matched` is 11, and `offlineArchiveTier` lists `noaa19_pass_2026_10_12.bin`. The JSON file is the audit record you would keep.

8.3 Clean up (or keep until tomorrow to see the rehydration finish)

An unlocked policy still blocks blob deletes while it is active, so the order is: delete the policy, delete the containers (management plane), then delete the account. If you keep the account overnight to watch the rehydration complete, run this tomorrow.

```
az storage container immutability-policy delete --account-name $SA -c satellite-archive -g
$RG --if-match "*"
for c in satellite-archive manifests cism-reference; do az storage container-rm delete --
storage-account $SA -g $RG -n $c --yes; done
az storage account delete -n $SA -g $RG --yes
```

Container delete through `container-rm` is a management-plane call and works with Contributor; the data-plane `az storage container delete --auth-mode login` would need a data role, which you removed in Part 5.

9. Debrief questions

Bring your worksheet answers to the whole-room debrief. The facilitator will collect the rehydration timings and the ABAC observations.

1. Where did the platform enforce a guarantee you rely on process for today? Name the control and the process it replaces.
2. How long did rehydration actually take, and what does that mean for researcher expectations when they ask for archived data?
3. Which ABAC condition would replace the most ACLs in your environment? Write the condition in words (role, action, attribute, value).
4. The lifecycle rule had to use creation time instead of last access time. What other design choice on this account made that necessary, and would you make the same trade for a research unit with active data?
5. If the policy had been locked at 36,500 days by mistake with the wrong scope, what would have been the consequence, and what in the lab protects against that?

Notes for the debrief

10. Worksheet

Item	Your value or observation
Storage account name	
Redundancy SKU	
Time of the rehydration request (UTC)	
Rehydration status at the end of the lab	
First line of the delete error in Checkpoint 2	
Result of the CISM download in Part 5	
Validation result and matched count	
Elapsed rehydration time at debrief	

11. Reference

- Store business-critical blob data with immutable storage (Microsoft Learn)
- Configure immutability policies for containers (Microsoft Learn)
- Example Azure role assignment conditions for Blob Storage (Microsoft Learn)
- Add or edit Azure role assignment conditions using Azure CLI (Microsoft Learn)
- Rehydrate an archived blob to an online tier (Microsoft Learn)

- Optimize costs by automatically managing the data lifecycle (Microsoft Learn)
- Manage and find Azure Blob data with blob index tags (Microsoft Learn)